

Total Recall: Agent Memory and Harness Engineering

WORKSHOP INSTRUCTIONS

workshopwaitingroom.com



JOIN THE WORKSHOP

Drop your GitHub handle below. We'll add you to `jasperan-org/agent-harness-workshop` with write access so you can run the workshop in Codespaces.

GITHUB HANDLE

jasperan



Nacho Martinez
@jasperan

VIEW →

REGISTER ME

YOU'RE IN.

You already have write access to @jasperan.

NEXT STEPS

1. Open the repo.
2. Open the repo: [jasperan-org/agent-harness-workshop](https://github.com/jasperan-org/agent-harness-workshop)
3. Hit CREATE CODESPACE below — it launches preconfigured (4-core · US West).

CREATE CODESPACE →

OPEN REPO →

[REGISTER ANOTHER]

jasperan-org / agent-harness-workshop

Q Type / to search

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

agent-harness-workshop

Public

Watch 0

Fork 0

Star 0

@jasperan has invited you to collaborate on this repository

View invitation

main

1 Branch

0 Tags

Go to file

Add file

<> Code

README

Total Recall — Agent Harness Workshop

Open in GitHub Codespaces

Build a **self-improving agent harness** from the ground up on a single **Oracle AI Database** — and watch the same harness running in your browser as you build it. The agent persists its own memory, grounds itself in your schema's meaning, turns the work it does into reusable **skills** and scheduled **automations**, and keeps its context window flat — all inside the database.

Who Am I?

Data Scientist Advocate @ **ORACLE**

Find my talks and projects on YouTube + GitHub

Launched two courses this year with Andrew Ng on Building Memory-Aware Agents



All Courses ▸ Short Course ▸ Building Adaptive AI Agents

Short Course

Intermediate

1h3m

Building Adaptive AI Agents

Instructors: Nacho Martínez, Casius Lee

ORACLE

✓ Earn an accomplishment with PRO

Enroll Now



AGENDA

What we'll talk *through*.

01 The agent stack

APPLICATION TO COMPUTE

02 The four shapes

FORM FACTOR, EVOLUTION

03 What is an agent

MODEL + HARNESS = AGENT

04 Inside the harness

THE LAYERS, ONE BY ONE

05 The memory substrate

FILESYSTEM + DATABASE

06 Find it, recall it

ENCODING, RETRIEVAL, MEMORY

07 Ground it, drive it, fit it

SEMANTIC, LOOP, CONTEXT

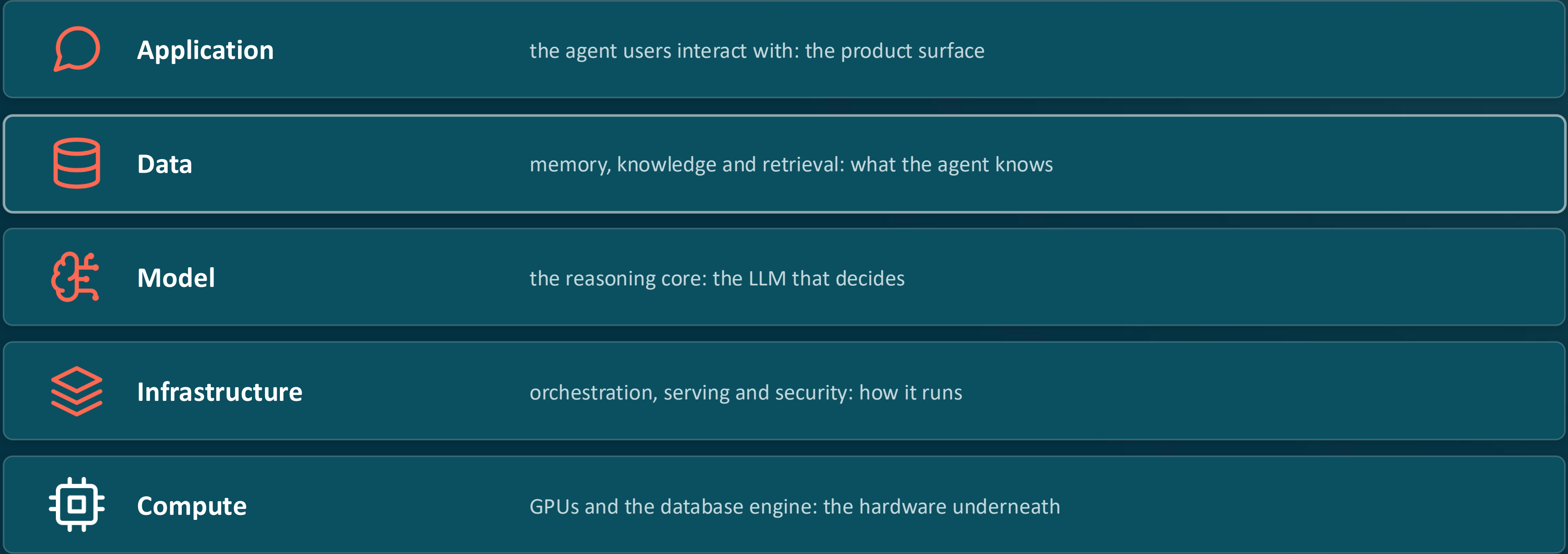
08 Continual learning

WEIGHT, LATENT, TOKEN SPACE

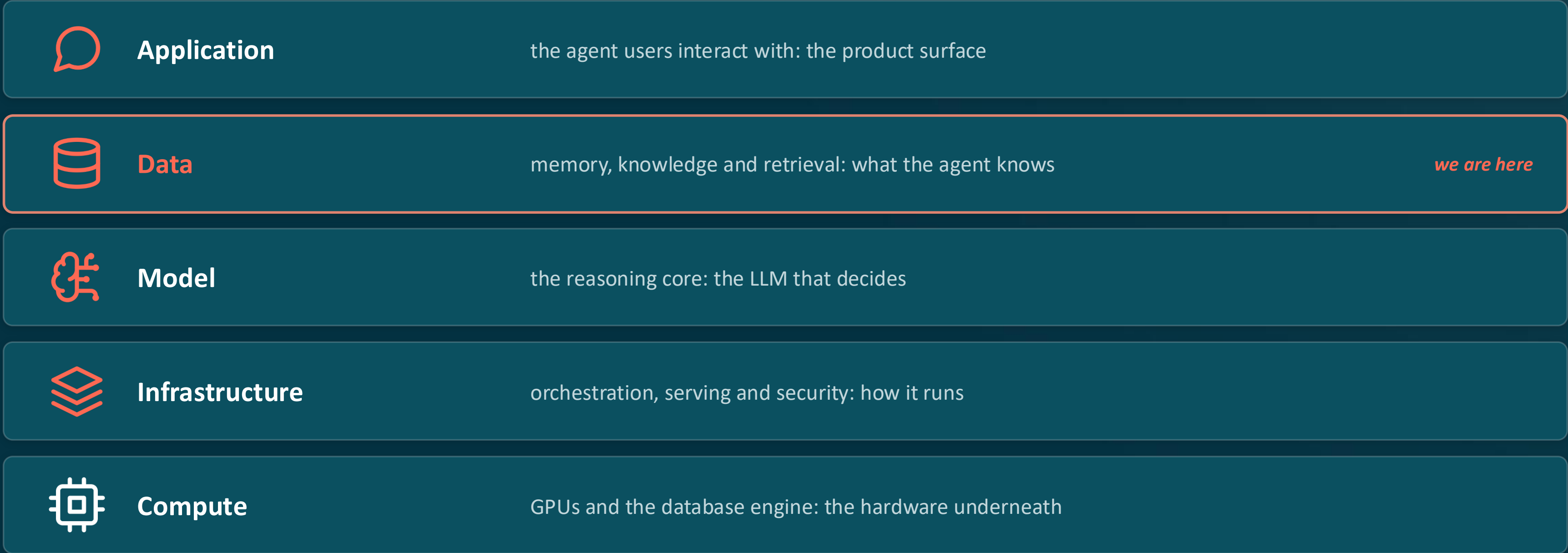
09 One substrate

IT ALL RUNS ON ORACLE

THE AGENT STACK



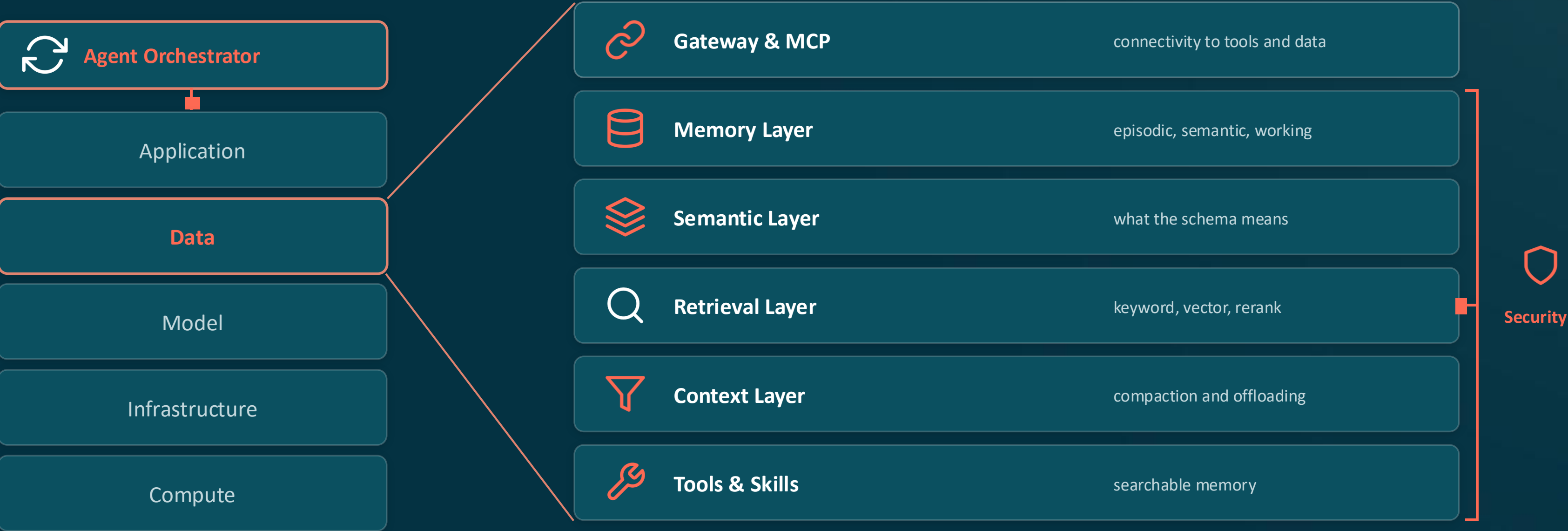
THE AGENT STACK



DOUBLE-CLICK THE DATA LAYER

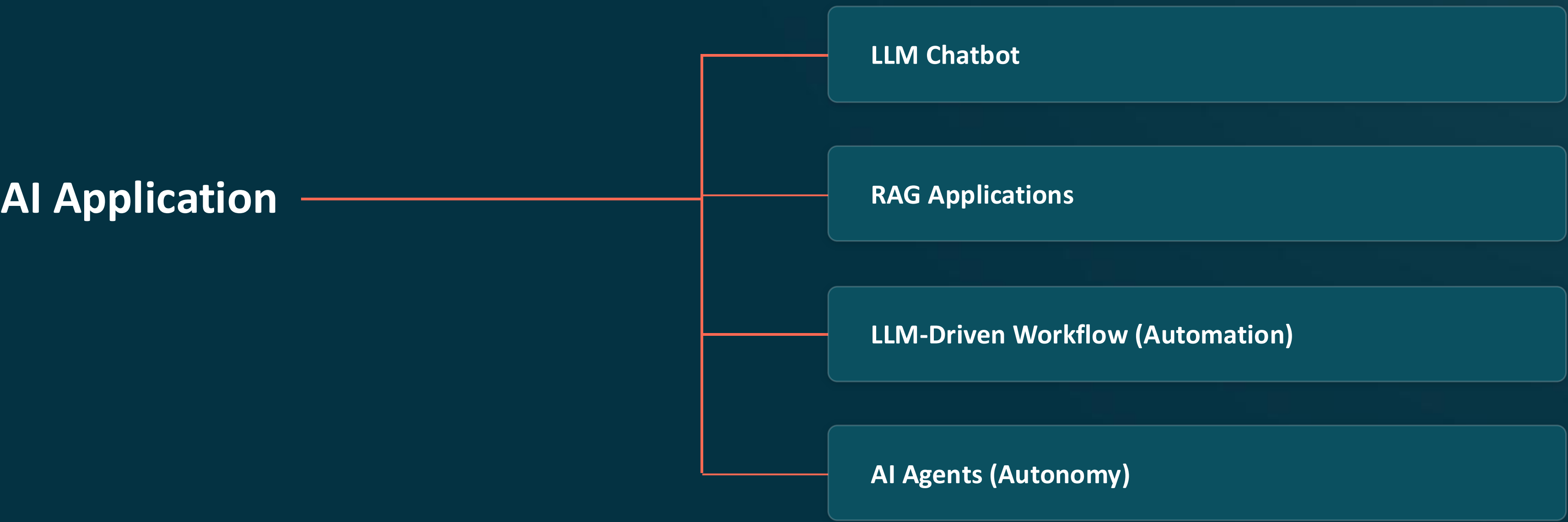
The data layer,*unpacked*.

Double-click the data layer: an orchestrator drives the app, a gateway fronts it, and memory, meaning, retrieval and context sit under one security boundary.



WHERE THE FIELD IS TODAY

Four shapes AI applications *take*.



WHAT THE AGENT DOES

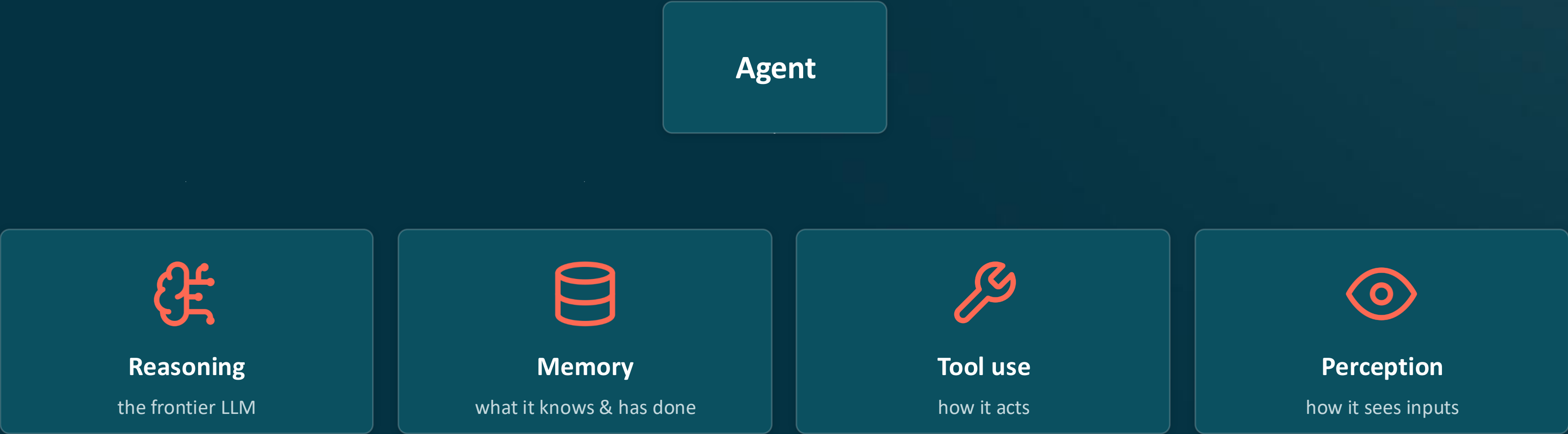
What is an **AI Agent**

Agent

An autonomous computational entity whose cognitive functions are powered by a **language model for reasoning**, augmented by a **database for memory**, extended through **tools for action**, and grounded in **inputs** that let it perceive its environment.

WHAT AN AGENT IS MADE OF

Reasoning, memory, tools, *perception*.



WHERE THE FIELD IS TODAY

Four shapes AI applications *take*.

AI Application

LLM Chatbot

RAG Applications

LLM-Driven Workflow (Automation)

AI Agents (Autonomy)

WHERE THE FIELD IS TODAY

Four shapes AI applications *take*.

AI Application

LLM Chatbot

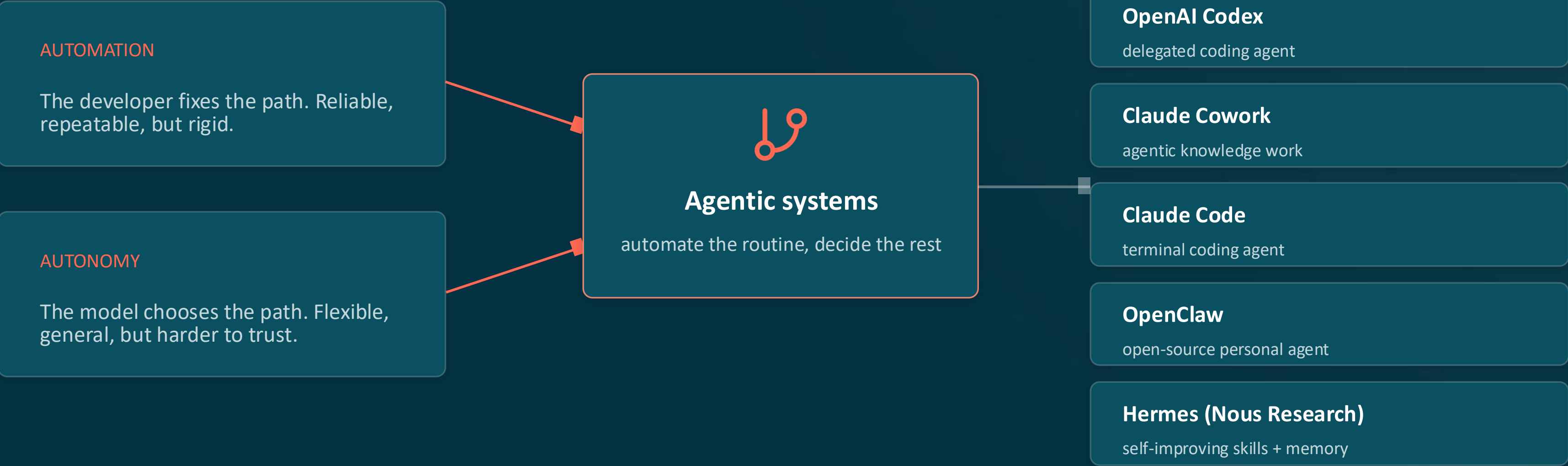
RAG Applications

LLM-Driven Workflow (Automation)

AI Agents (Autonomy)

WHERE THE FIELD HEADS

Automation and autonomy *converge*.



The frontier products are already here: agents that automate what is routine and decide what is not.

AGENT HARNESS

What is an **agent harness**



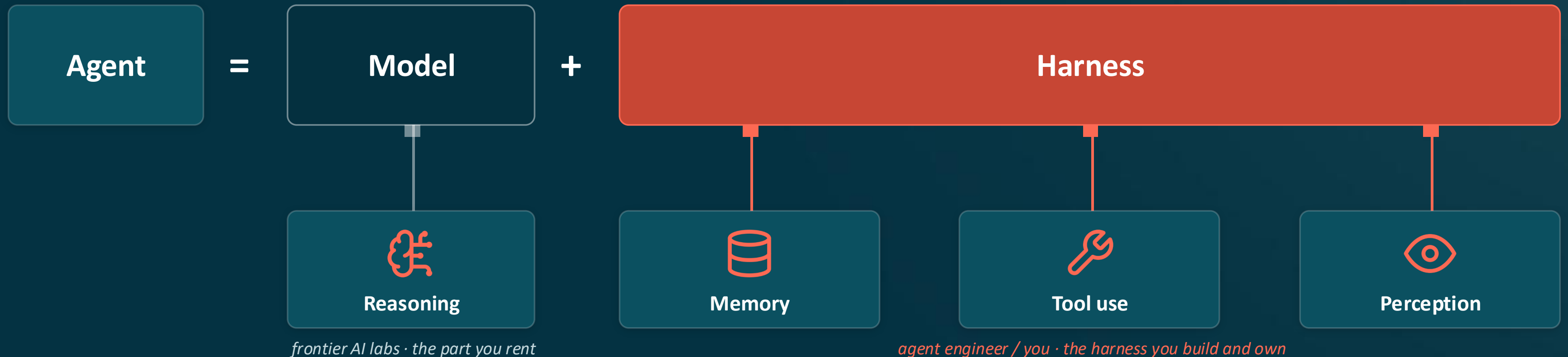
The **engineering scaffolding** around a model and its agent loop: the **memory, retrieval, tools, context and control** layers that turn a capable but stateless LLM into a system which produces **reliable, repeatable outcomes**, run after run.

Why it matters the model supplies raw capability; the harness is what makes that capability dependable.

Agent = model + harness.

THE REFRAMING

Components of a Dependable System



Reasoning is the part you rent. Memory, tools and perception are the harness you build, along with the other surfaces that support them (prompts, retrieval, sandbox, observability, and more).

Guidance. A good agent harness produces reliable, predictable output regardless of which model you swap in. Memory, tools, and perception are yours to engineer; reasoning is the part you rent.

INSIDE THE HARNESS

We'll dissect it *layer by layer*.

AGENT HARNESS

everything inside is what you build and own

- | | | |
|---|---------------------------------|---|
| 1 | Model layer | the frozen reasoning core the harness is built around |
| 2 | Storage layer | where memory physically lives: files and the database |
| 3 | Encoding & retrieval | turn text into meaning, hand back the slice |
| 4 | Memory layer | episodic, semantic, working, procedural |
| 5 | Semantic layer | teach the agent what your schema means |
| 6 | Agent loop | the typed graph that drives every layer above |
| 7 | Context engineering | keep the window dense as the session grows |

memory engineering

memory engineering

memory engineering

MODEL LAYER

The frozen *reasoning* core.

The frontier LLM is the reasoning engine, and the one part of the system we do not build. **It is a frozen utility: identical from one turn to the next.** Everything the harness does is built around it, and everything that makes the agent improve is the harness, not the model.

Reasoning, out over the network

One chat LLM, provider-toggleable, the only outbound call at runtime.

Two models, in the database

An embedder and a cross-encoder reranker run inside the engine (ONNX).

Frozen, by design

Weights never change at runtime; the harness is what learns.

On Oracle reasoning is hosted on OCI Generative AI; embeddings and reranking run in-database.

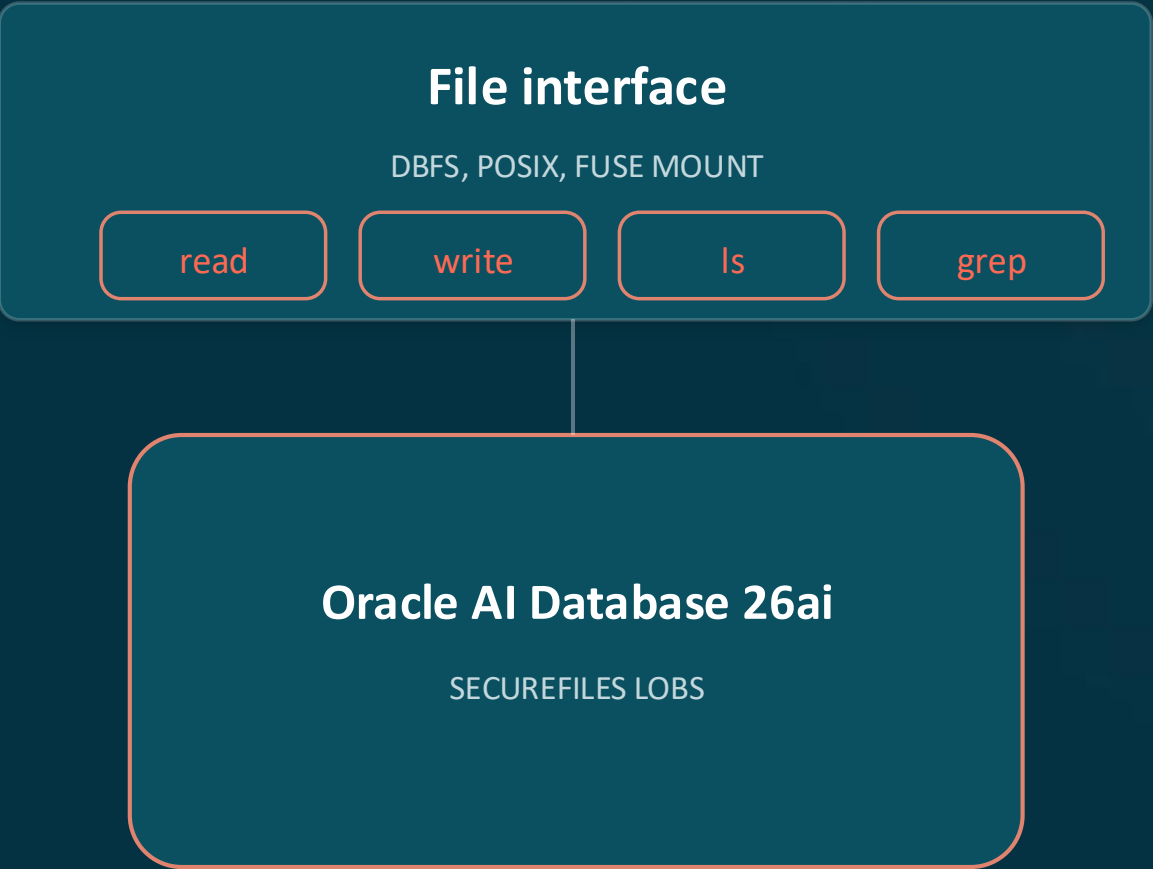
THE RESOLUTION

Oracle Database File System: *the best of both worlds.*

A filesystem interface agents already speak, backed by Oracle AI Database 26ai.

WHAT FILES GIVE YOU

- On-distribution verbs
- Legible & versionable
- Agent-controlled I/O
- Hierarchical paths



WHAT THE DATABASE ADDS

- ACID + HA (Data Guard)
- AI Vector Search
- Relations + ranking
- Scale, security, backup

Hybrid in practice: a scratchpad on the filesystem holds short-term working memory; a scheduled pipeline **promotes** what is worth keeping into long-term memory in tables. One pipeline, one ACID boundary.

ENCODING & RETRIEVAL

Turn text into meaning, hand back the right *slice*.

WRITE PATH, ENCODING

Embeddings are produced **inside the database** by a loaded ONNX model, reached through **OracleVS**.



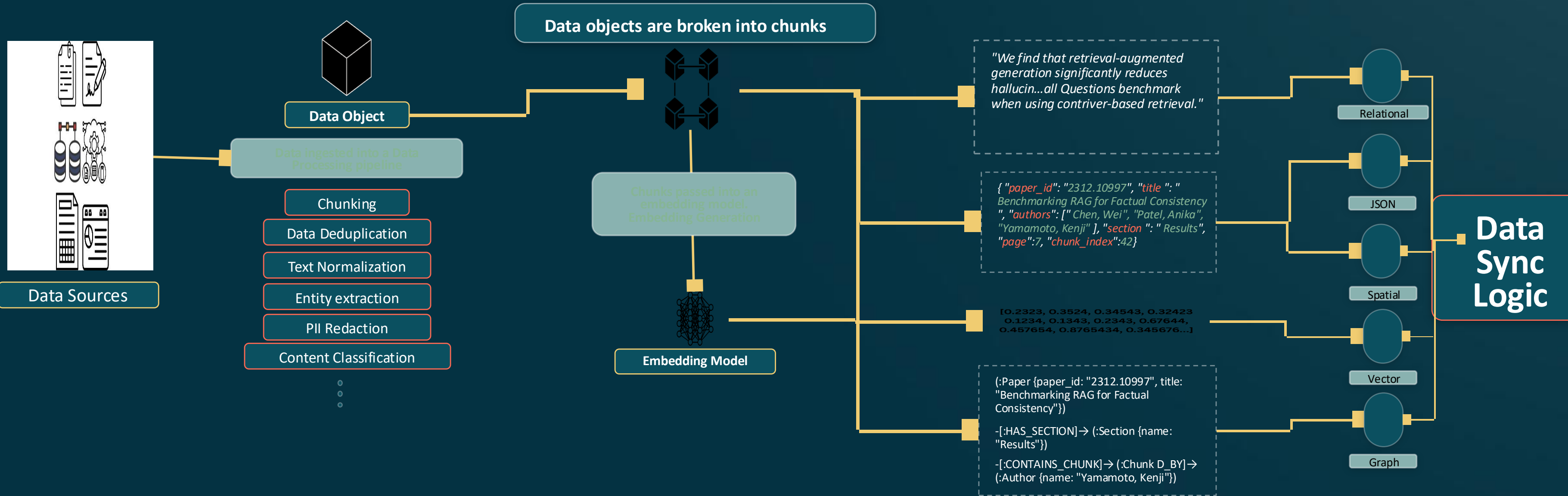
One model, one vector space. Nothing leaves the engine to be vectorised.

READ PATH, THE RETRIEVAL LADDER

- | | | |
|---|---------------------|--|
| 1 | Keyword | Oracle Text CONTAINS, exact terms & identifiers |
| 2 | Vector | cosine over HNSW; query embedded in the same space |
| 3 | Hybrid (RRF) | fuse keyword + vector; no weight tuning |
| 4 | Rerank | cross-encoder reads query & document together |

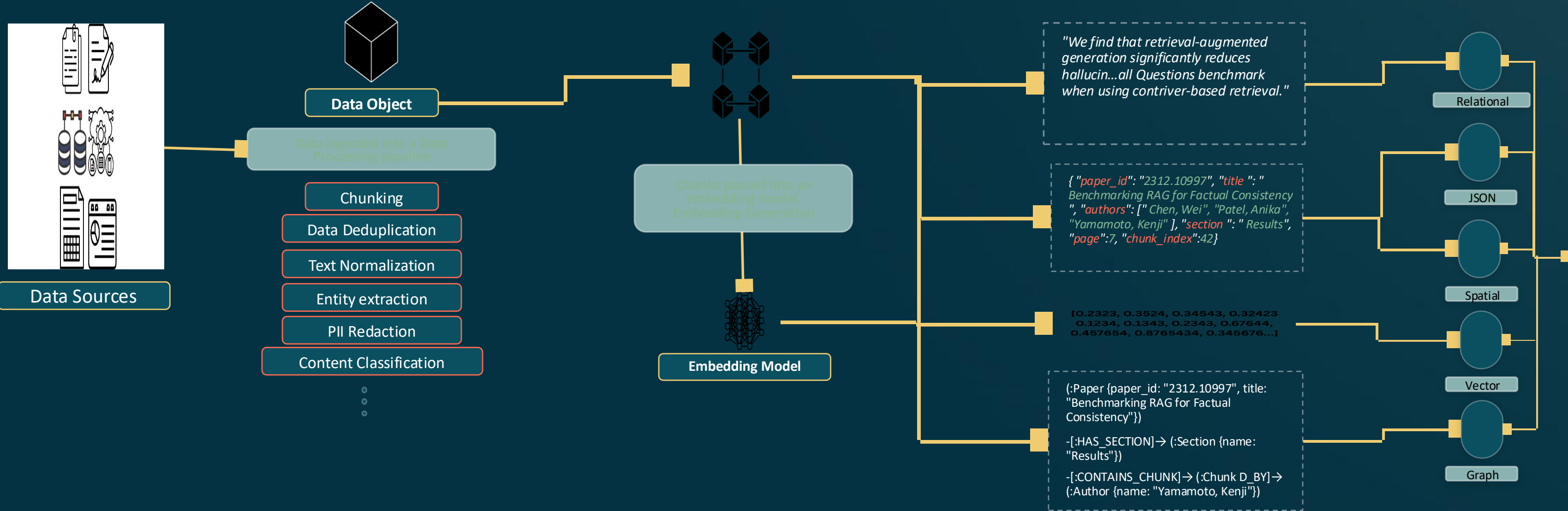
Retrieval is *progressive disclosure for tables*: hand the model the few rows it needs, nothing more.

Specialized Systems Require *Dedicated Attention*

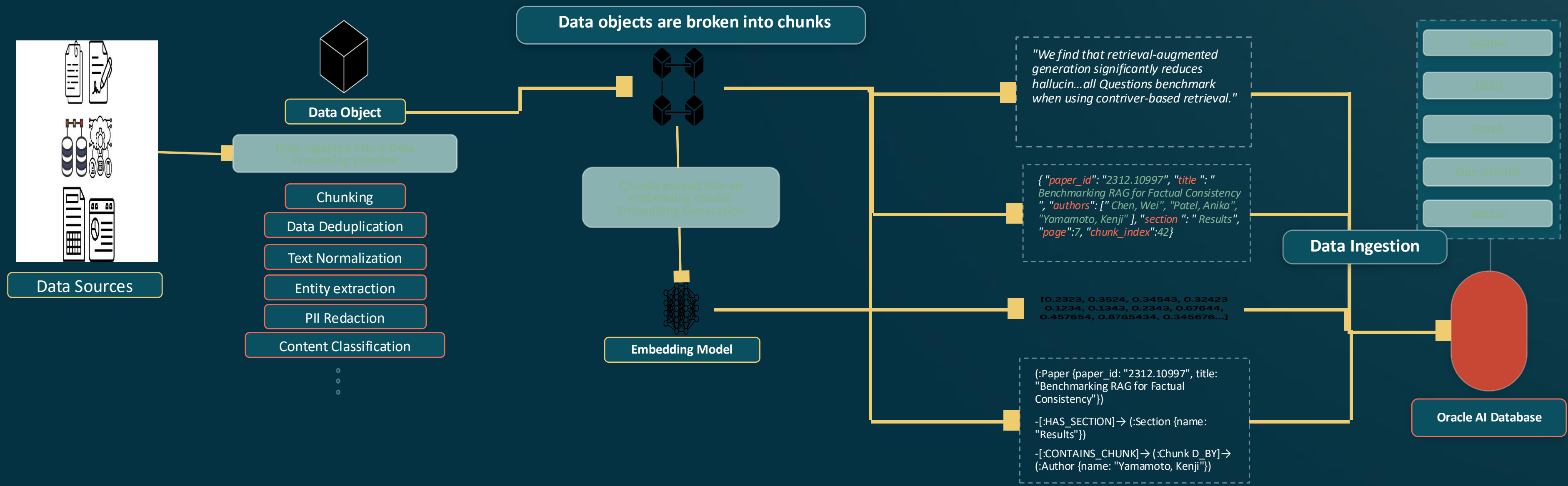


We are seeing Increase in *Cognitive Load*

Anti Pattern

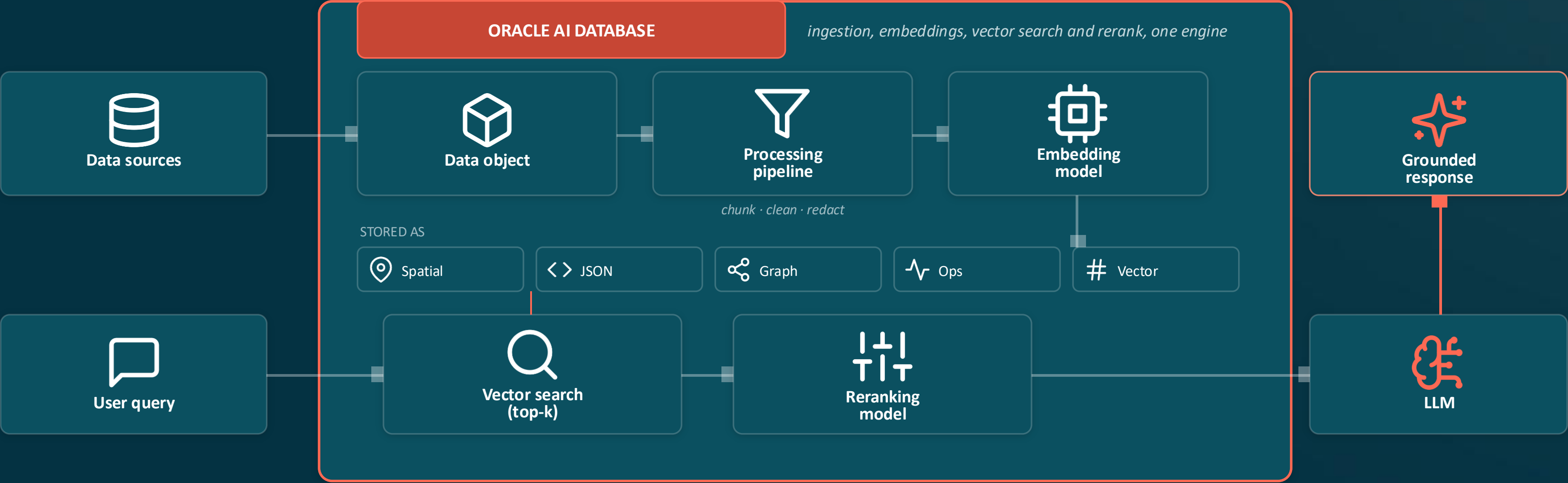


Promoting and championing *Cognitive Focus*



HOW THE PIECES CONNECT

Oracle is the *engine*, not a step.



Only the model (OCI Generative AI) and the I/O sit outside. Everything else is the database.

AGENT MEMORY

What is **agent memory**



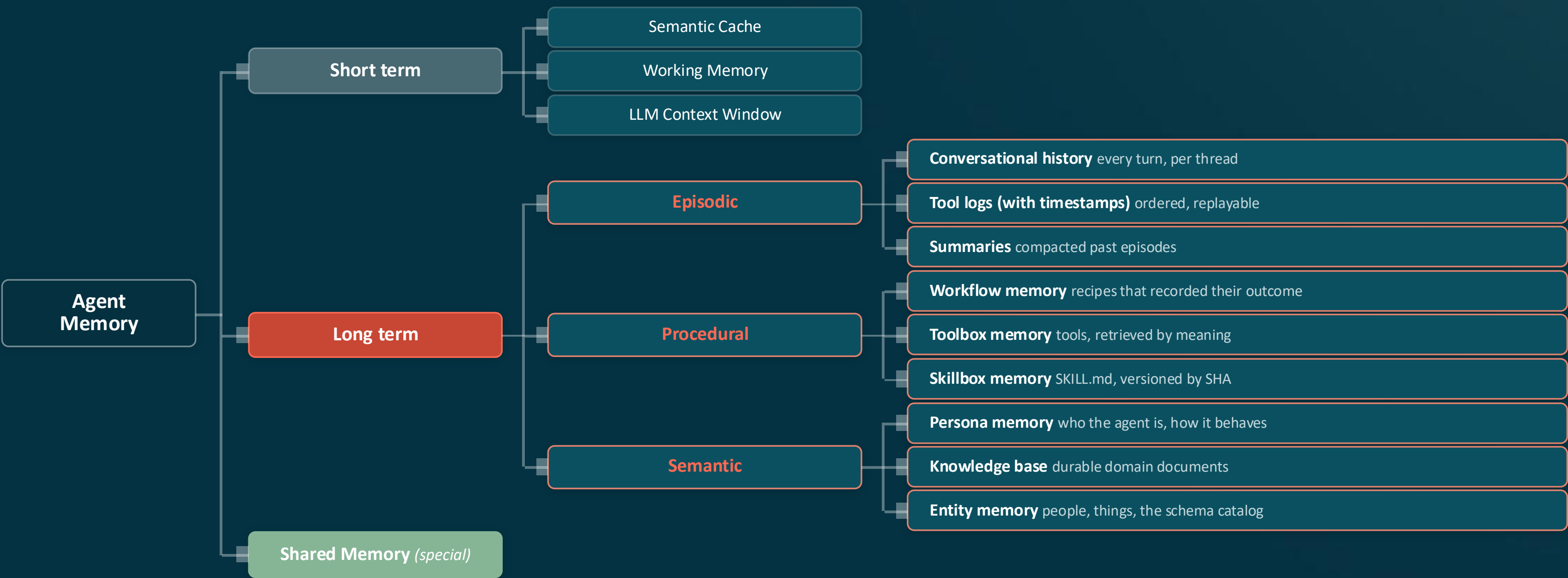
The conceptual description of the **mechanisms and systems** that enable an agent to **retain, reuse, refine, and recall** information, for **adaptability and continual learning**.

Why it matters: without memory, an agent has amnesia. Memory is what turns a stateless model into a persistent entity.

EVERY KIND, AND WHERE IT LIVES

Agent memory is *not one thing*.

Short term is ephemeral, long term persists across sessions, and shared memory spans agents. Naming the types tells you where each kind should live.



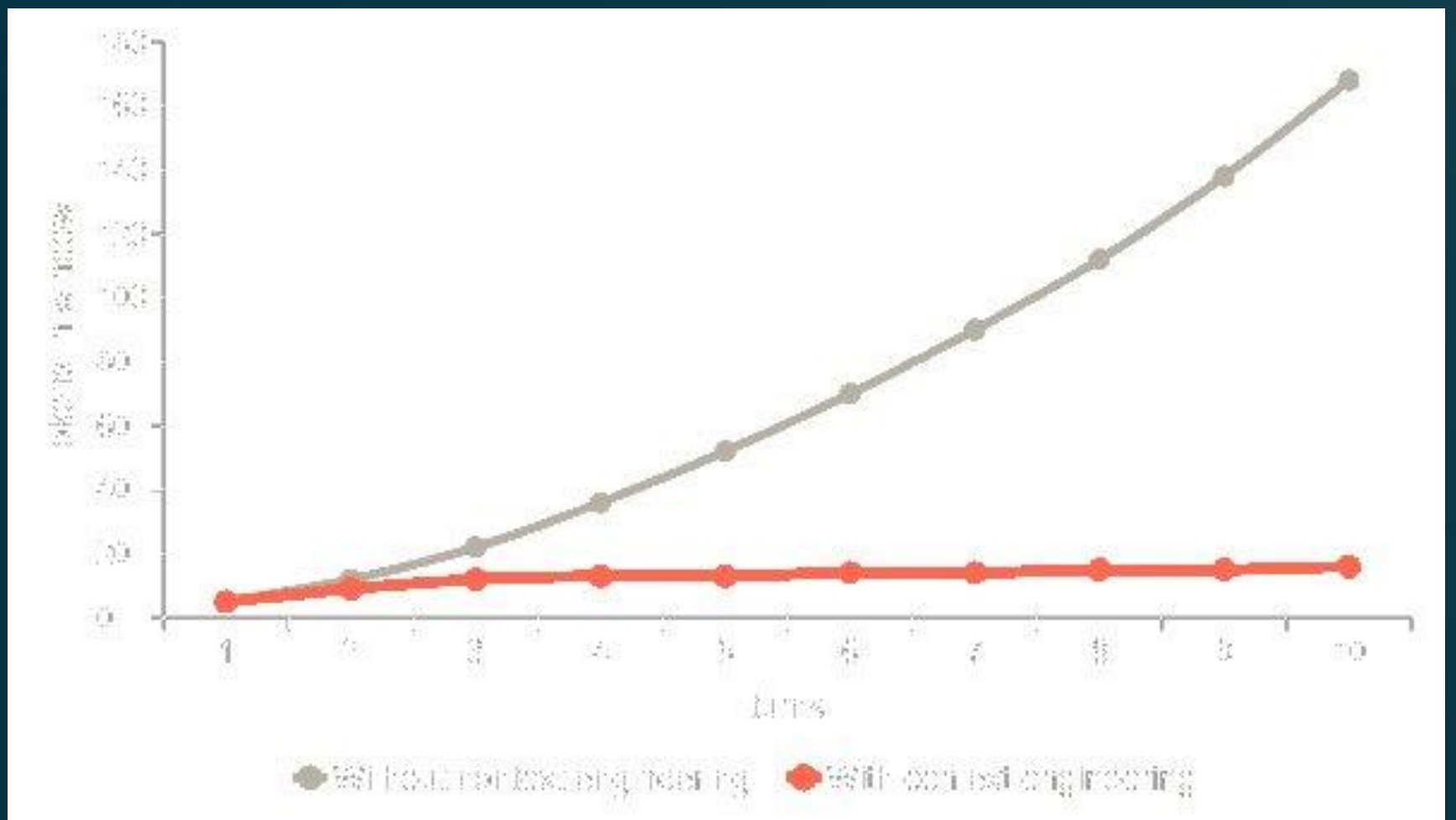
CONTEXT ENGINEERING

A bigger window is not the *fix*.

Context rot. Model quality slips before the advertised limit, and the relevant fact gets harder to attend to as inputs grow. Attention, not capacity, is the budget, so you compact *ahead* of the limit, not at it.

Compaction replace a long history with a faithful summary (the OAMP context card).

Offloading move a large payload out of the window, leave a pointer; substrate-aware (table or file).



Measured, not asserted: flat context versus blow-up.

MEMORY ENGINEERING

What is **Memory Engineering**



The discipline of **designing, building, and operating the systems that give an agent memory** : the schemas, indexes, retrieval, lifecycle, and governance that let it **retain, recall, and refine knowledge reliably at scale**.

Why it matters most teams build a model and a prompt, then bolt memory on. Memory engineering treats it as a first-class discipline, not an afterthought.

MEMORY LAYER

Agent memory is a *discipline*.

Schemas, indexes, lifecycle, retrieval, governance: the harness manages all of it.

SCHEMAS

Memory types

episodic · semantic
procedural · working

INDEXES

Make it findable

vector · lexical
graph · hybrid

LIFECYCLE

Write, evict, refresh

expiry · summarisation
compaction

RETRIEVAL

Read in context

top-k · fusion
rerank

GOVERNANCE

Scope by identity

user · agent
thread · RLS

ORACLE'S ANSWER, OAMP

Oracle AI Agent Memory Package

`pip install oracleagentmemory`

01
memory
facts, corrections, events

02
thread
scoped conversations

03
scratchpad
ephemeral working state

WHAT OAMP ABSORBS

One line instead of a memory *subsystem*.

WITHOUT OAMP · YOU CARRY THE LOAD

-  Decide when to summarise
-  Write a summariser: keep signal, drop filler
-  Rank + retrieve relevant memories per turn
-  Extract durable facts from chatty turns
-  Glue summary + memories + recent into a prompt
-  Bound it all to a token budget

OAMP
absorbs this

WITH OAMP · ONE CALL

`mem.context_card(thread_id)`

BOUNDED, PROMPT-READY BLOCK

-  **Topics**
-  **Summary**
-  **Relevant memories**
-  **Recent turns**

The **developer** writes one line instead of a memory subsystem.

The **agent** spends attention on the task, not on housekeeping its own memory.

THE CONTEXT CARD

What the agent actually *reads*.

```
<context_card>
<topics> q3 growth · outdoors · business risk </topics>
<summary>
Reviewing Q3 performance. Growth driven by
Outdoors, +12% QoQ. User now asks the main
risk to that driver.
</summary>
<relevant_information>
<fact> Outdoors drove Q3, +12% QoQ </fact>
<preference> prefers metric units </preference>
<memory> risk question asked, unanswered </memory>
</relevant_information>
<recent_messages>
user What drove Q3 growth?
assistant The Outdoors category, +12% QoQ.
user What is the main risk to that?
</recent_messages>
</context_card>
```

WHAT EACH PART IS DOING

<topics>
A semantic fingerprint of the thread: cheap orientation and routing.

<summary>
A running recap that also captures the current intent, not just history.

<relevant_information>
Typed and timestamped. A fact is truth; a preference (metric units) shapes the answer; an episodic memory flags the question is still open.

<recent_messages>
The last turns verbatim, with role, for local coherence.

AGENT LOOP

What is the **agent loop**

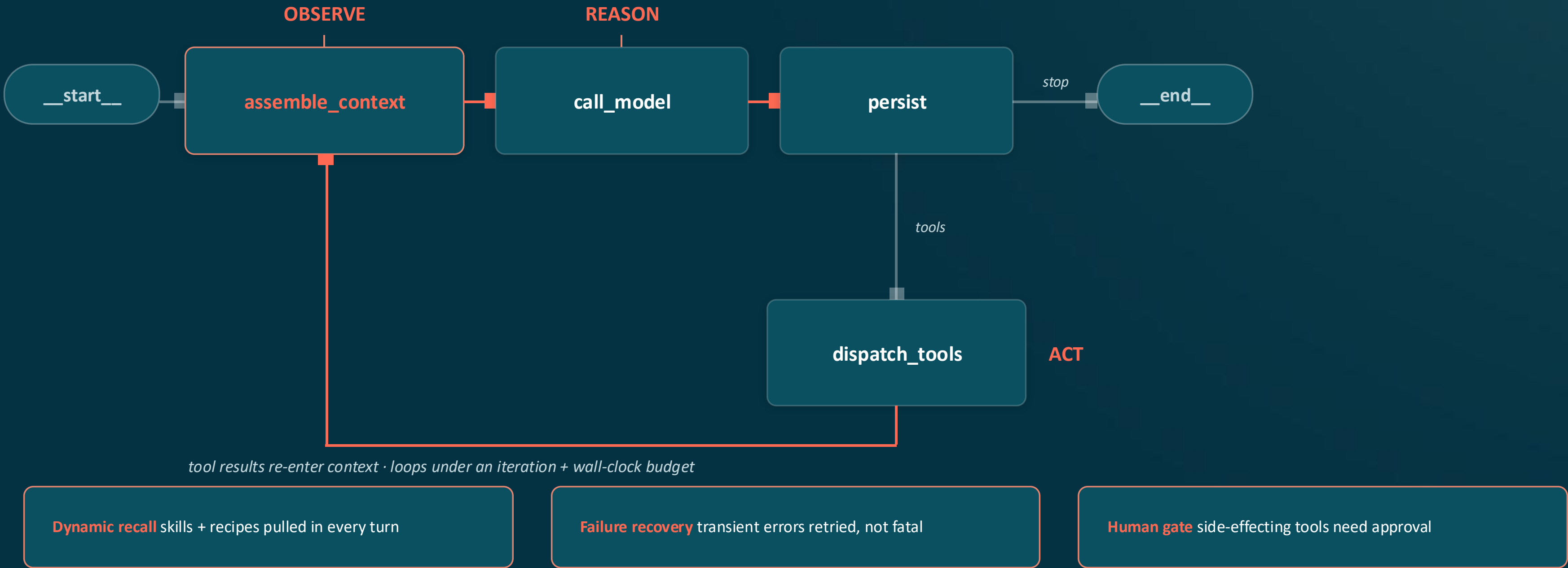


The control structure that drives an agent: **assemble context, call the model, run the tools run the tools it asks for, persist what happened, and decide whether to continue** , under **continue** , under an explicit budget.

Why it matters the loop is the component that turns a frozen model into something that acts, recovers, and improves.

AGENT LOOP

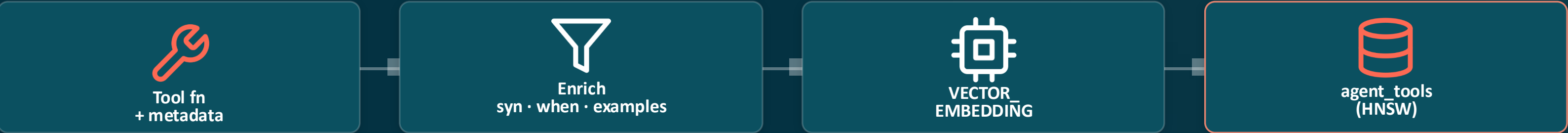
The graph that *drives* every layer.



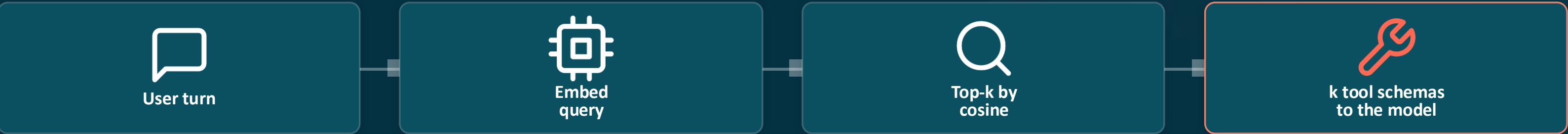
THE TOOLBOX PATTERN

Only the tools that *fit the turn*.

WRITE PATH, REGISTER A TOOL



READ PATH, EVERY USER TURN



The registry grows past 30 tools without bloating the prompt. Each tool is stored with an enriched document (synonyms, when-to-use, examples) and its embedding; the callable stays in Python, only the schema and vector live in the database.

ORACLE MCP SERVERS

Tools and MCPservers

	July 2026	Oracle's first MCP server, shipped in July	MILESTONE
	~30 sample servers	MySQL · HeatWave · JDBC · OCI cost, Terraform	ONE REPO
	Two server shapes	OCI-managed, or HTTPS streaming self-run	CLOUD & ON-PREM
	Federated identity	OAuth 2 across Entra ID · Okta · Auth0	OCI-MANAGED
	Identity in the session	roles reach the session, redaction still applies	DEEP DATA SECURITY

The agent is just another client — and it queries as a regular user (and inherits its roles & permissions)

REPORTS, NOT RAW SQL

Let agents run reports, not queries

NATURAL LANGUAGE → SQL

A different query every time

the same question comes back as another statement

Relationships get guessed

joins and grain are inferred from names, not known

Prompt engineering as upkeep

every schema change becomes a prompt change

TRUSTED REPORTS AS TOOLS

Upload what analysts wrote

existing reports, hard-coded queries and all

The agent selects, then binds

pick the report, supply the inputs, run it

The same answer, every turn

consistent results the business signed off on

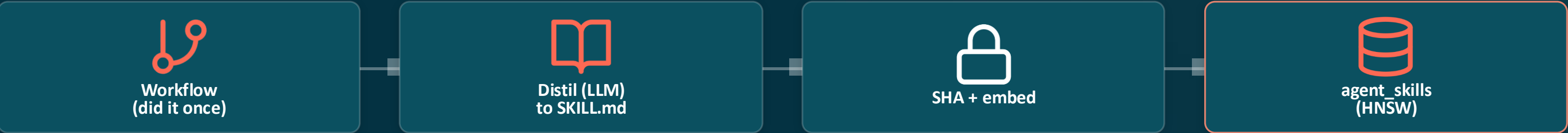
Reports = established, robust queries

MCP is how a tool arrives. The toolbox is how it gets **chosen**

THE SKILLBOX PATTERN

Distil once, recall *on demand*.

AUTHOR A SKILL



TWO-LEVEL RETRIEVAL

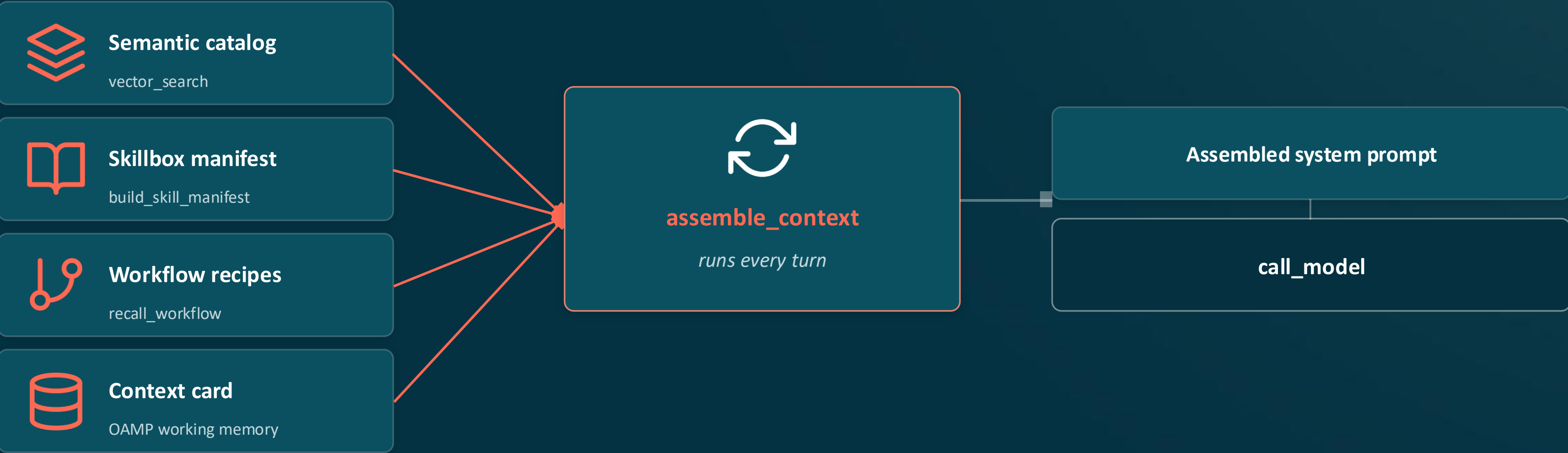
Level 1 a one-line name: description manifest, retrieved every turn so the agent knows which skills exist.

Level 2 load_skill pulls the full SKILL.md body, only when the agent decides to use it.

***SHA-256 versions each skill:** refresh from a source only when the hash changes, and promoting a workflow retires it from recall so the agent never sees both.*

DYNAMIC SKILL DISCOVERY

Re-discovered *every turn.*



Each turn queries the skillbox against the latest user message. A newly learned skill shows up automatically on the very next relevant turn, no redeploy.

CONTINUAL LEARNING

What is **continual learning**



The ability of a system to **keep getting better from experience after training** : acquiring skills, updating beliefs, and correcting mistakes, **without a full retrain**.

Why it matters it is the biggest gap between today's agents and human intelligence. The question is where the new knowledge is stored.

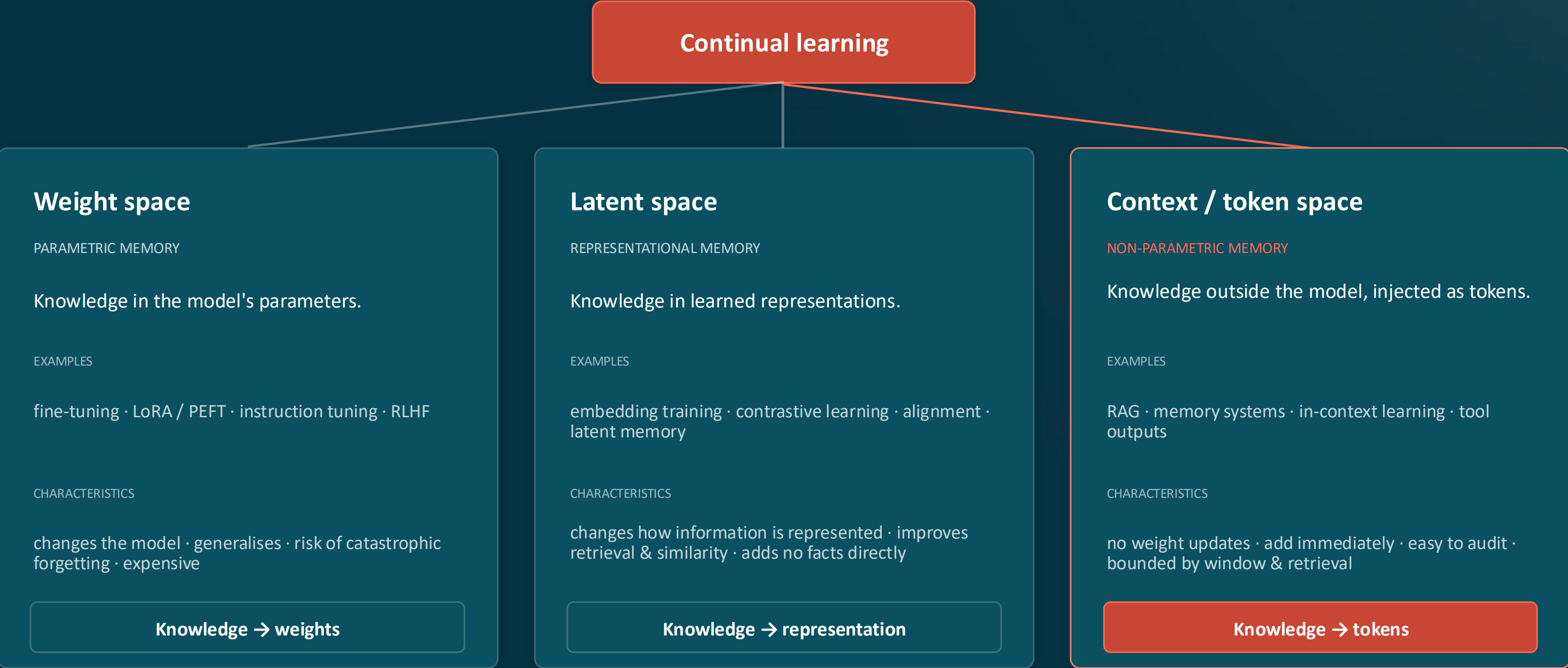
CONTINUAL LEARNING

How does a *frozen* model get better?

The model's weights never change at runtime. Yet the agent improves. The trick is to choose where new knowledge lives: in the **weights**, in the **representations**, or in the **context** the model reads.

WHERE KNOWLEDGE LIVES

Continual learning *branches three ways.*



A MORE COMPLETE TAXONOMY

One taxonomy, three *memories*.

Continual learning =

parametric memory
(weights)

+

representational memory
(latents)

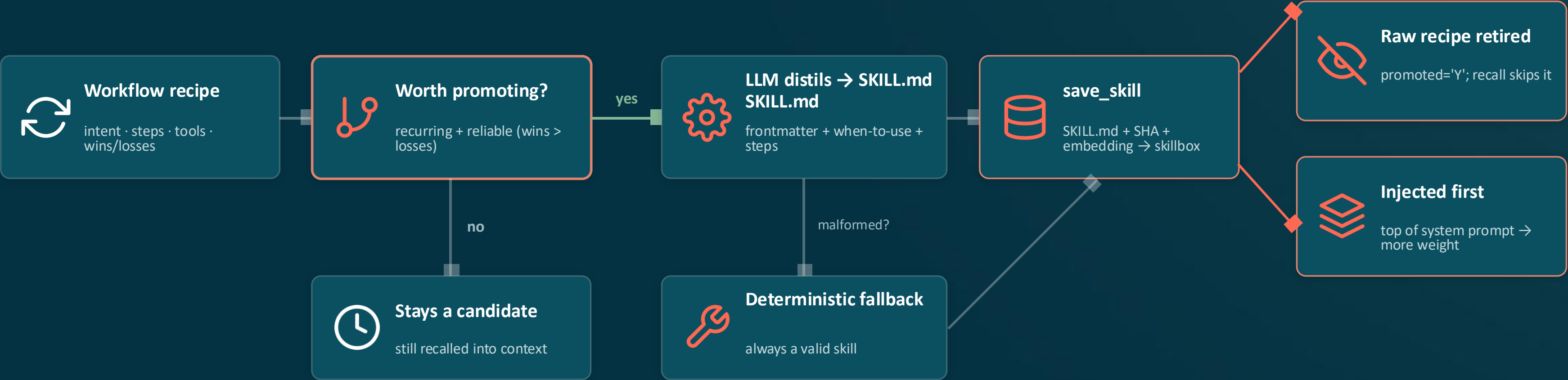
+

non-parametric memory
(tokens / retrieval)

Our agent lives in the third column: non-parametric memory.

PROMOTE TO A SKILL

Distil a recipe(*workflow memory*) into reusable *skills*

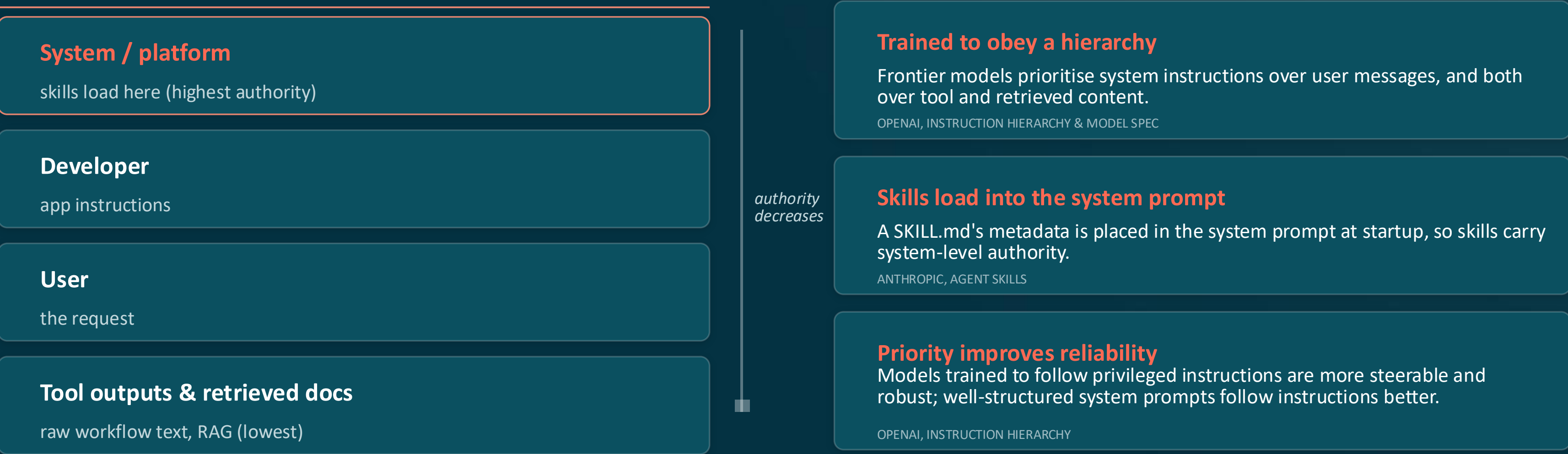


The two outcomes on the right are the point. The raw recipe is retired so the agent does not spend attention on both, and the skill is injected at high authority, so the same knowledge counts for more at inference.

INSTRUCTION HIERARCHY

Skills sit in the *system* layer.

INSTRUCTION HIERARCHY

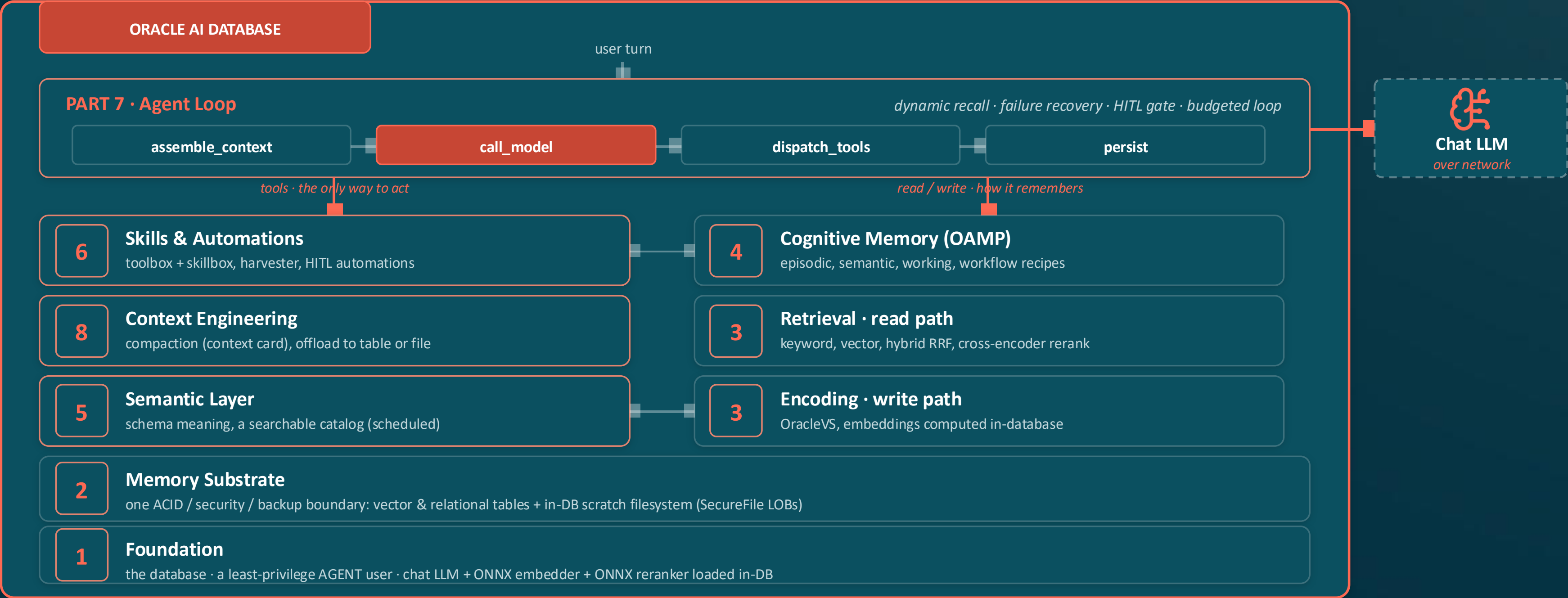


Put durable, must-follow knowledge in skills (system layer); keep volatile data in retrieval (lower layer).

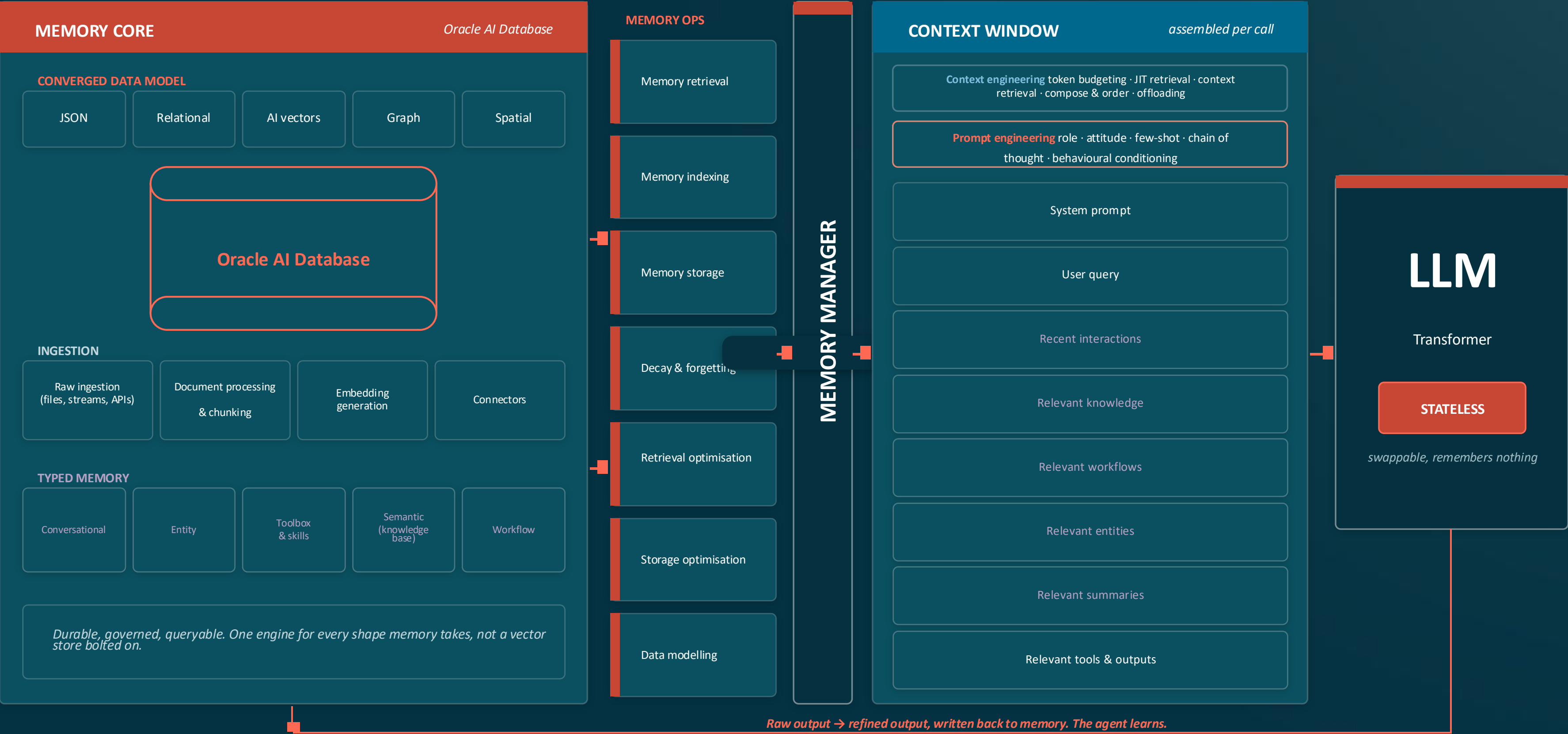
TOTAL RECALL WORKSHOP

The whole harness,*one engine.*

Read bottom-up, the order we build it. Only the chat model leaves the database.



The memory core is the last line of defence



THE HARNESS ON ORACLE

Every layer maps to an Oracle *primitive*.

AGENT HARNESS ON ORACLE	
Model	OCI Generative AI (hosted frontier models) + in-database ONNX embedder & reranker
Storage	In-database POSIX filesystem (SecureFiles LOBs) + relational / vector tables, one ACID boundary
Encoding & retrieval	OracleVS + in-DB embeddings, AI Vector Search (HNSW), Oracle Text, hybrid RRF, rerank
Memory	Oracle AI Agent Memory Package (OAMP)
Semantic	Comments + annotations + use-case domains + views + V\$SQL, catalog (DBMS_SCHEDULER)
Skills & automations	Toolbox / skillbox tables (HNSW), SHA-versioned SKILL.md, DBMS_SCHEDULER automations
Agent loop	LangGraph state graph + Oracle-backed checkpoints
Context engineering	OAMP context card (compaction) + substrate-aware offloading, row-level security

Join the Oracle AI Developer Discord Channel to ask questions about Agent Memory and the Oracle AI Agent Memory Package



Oracle AI Developer Hub

